

多分类与TensorFlow

学习目标

Softmax回归

交叉熵损失

框架使用

案例：Mnist手写数字识别神经网络实现

数据集介绍

特征值

目标值

Mnist数据获取API

网络设计

全连接层计算

前期确定事情与流程

主网络搭建流程

完善模型功能

如何计算准确率

增加变量tensorboard显示

增加模型保存加载

增加模型预测结果输出

调整学习率调整网络参数带来的问题

总结

学习目标

- 目标
 - 知道softmax回归的原理
 - 应用softmax_cross_entropy_with_logits实现softmax以及交叉熵损失计算
 - 应用matmul实现多隐层神经网络的计算
- 应用
 - 应用TensorFlow完成Mnist手写数字识别

到目前为止，我们所接触的都是二分类问题，神经网络输出层只有一个神经元，表示预测输出 $\hat{y}$ 是正类的概率 $P(y = 1|x)$ ， $\hat{y} > 0.5$ 则判断为正类，反之判断为负类。那么对于多分类问题怎么办？

Softmax回归

对于多分类问题，用N表示种类个数，那么神经网络的输出层的神经元个数必须为 $L[\text{output}] = N$ ，每个神经元的输出依次对应属于N个类别当中某个具体类别的概率，即 $P(y = N_1|x), \dots, P(y = N_n|x)$ 。

输出层：

$Z^{[L]} = W^{[L]}a^{[L-1]} + b^{[L]}$ ，Z的输出值个数为类别个数

需要对所有的输出结果进行一下softmax公式计算：

$a_i^{[L]} = \frac{e^{Z_i^{[L]}}}{\sum_{i=1}^C e^{Z_i^{[L]}}}$ ，并且满足 $\sum_{i=1}^C a_i^{[L]} = 1$ ，我们来着一下计算案例：

$$Z^{[L]} = \begin{bmatrix} 5 \\ 2 \\ -1 \\ 3 \end{bmatrix} \quad t = \begin{bmatrix} e^5 \\ e^2 \\ e^{-1} \\ e^3 \end{bmatrix}$$

softmax公式得来

概率：

$$g^{[L]}(z^{[L]}) = \begin{bmatrix} \frac{e^5}{(e^5 + e^2 + e^{-1} + e^3)} \\ \frac{e^2}{(e^5 + e^2 + e^{-1} + e^3)} \\ \frac{e^{-1}}{(e^5 + e^2 + e^{-1} + e^3)} \\ \frac{e^3}{(e^5 + e^2 + e^{-1} + e^3)} \end{bmatrix} = \begin{bmatrix} 0.842 \\ 0.042 \\ 0.002 \\ 0.114 \end{bmatrix}$$

所有值加起来等于1、每个输出都能得出一个小于1的概率值

交叉熵损失

对于softmax回归（逻辑回归代价函数的推广，都可以称之为交叉熵损失），它的代价公式为：

$$L(\hat{y}, y) = - \sum_{j=1}^C y_j \log \hat{y}_j$$

总损失函数可以记为 $J = \frac{1}{m} \sum_{i=1}^m L(\hat{y}, y)$

逻辑回归的损失也可以这样表示，：

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) + y^{(i)} \log h_{\theta}(x^{(i)}) \right]$$

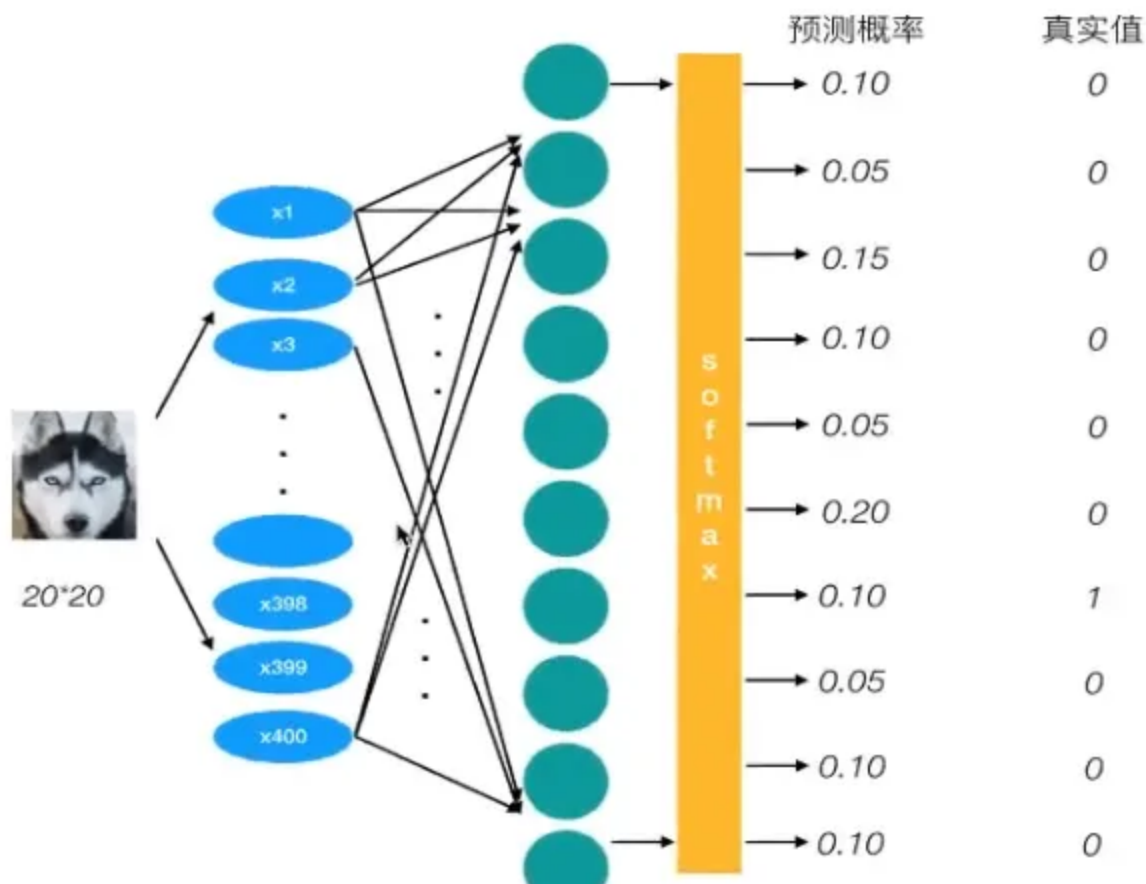
目标值
↑ 预测属于1的概率
目标值是1
属于1概率

$$= -\frac{1}{m} \left[\sum_{i=1}^m \sum_{j=0}^1 1 \{y^{(i)} = j\} \log p(y^{(i)} = j | x^{(i)}; \theta) \right]$$

预测的概率=0, 1
=1的概率
=0类别的概率

所以与softmax是一样的，一个二分类一个多分类衡量

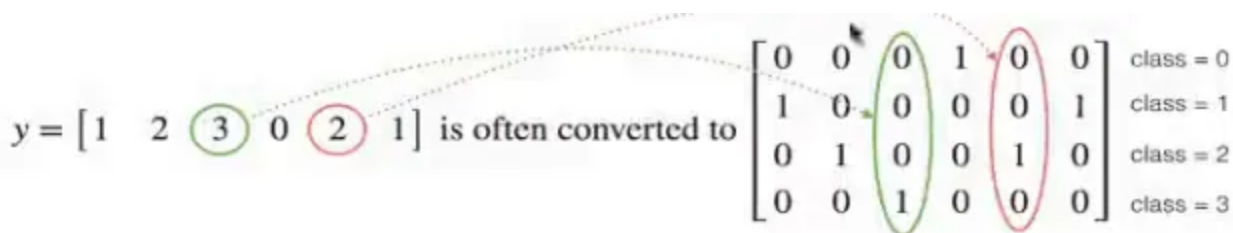
对于真实值会进行一个one-hot编码，每一个样本的所属类别都会在某个类别位置上标记。



上图改样本的损失值为：

$$-log(0.10) - log(0.05) - log(0.15) - log(0.10) - log(0.05) - log(0.20) - log(0.10) - log(0.05) - log(0.10) - log(0.10)$$

注：关于one_hot编码



框架使用

便于编程：包括神经网络的开发和迭代、配置产品

运行速度：特别是训练大型数据集时

目前最火的深度学习框架大概是Tensorflow了。Tensorflow框架内可以直接调用梯度下降算法，极大的降低了编程人员的工作量。

案例：Mnist手写数字识别神经网络实现

数据集介绍



数据集文件：

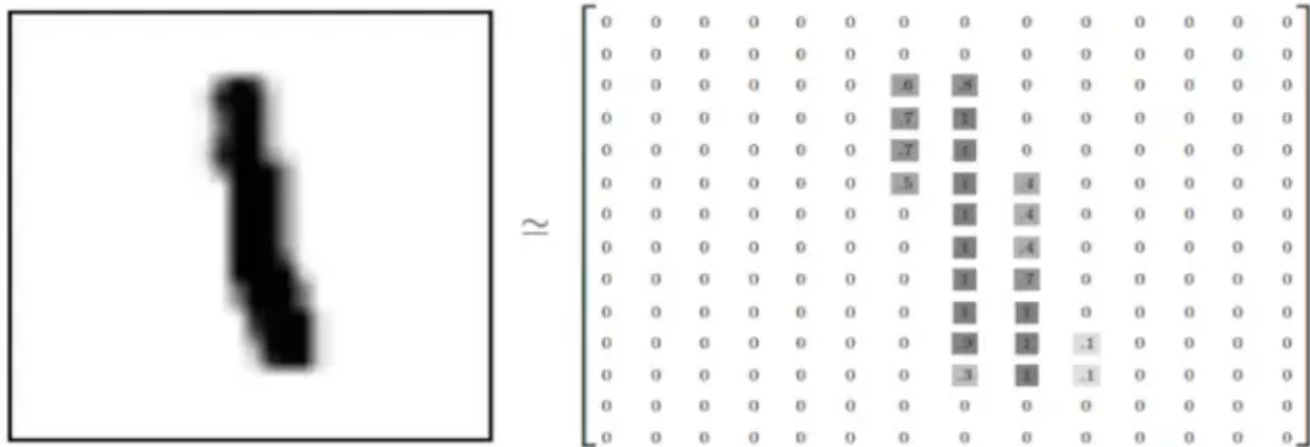
文件说明：

- train-images-idx3-ubyte.gz: training set images (9912422 bytes)
- train-labels-idx1-ubyte.gz: training set labels (28881 bytes)
- t10k-images-idx3-ubyte.gz: test set images (1648877 bytes)
- t10k-labels-idx1-ubyte.gz: test set labels (4542 bytes)

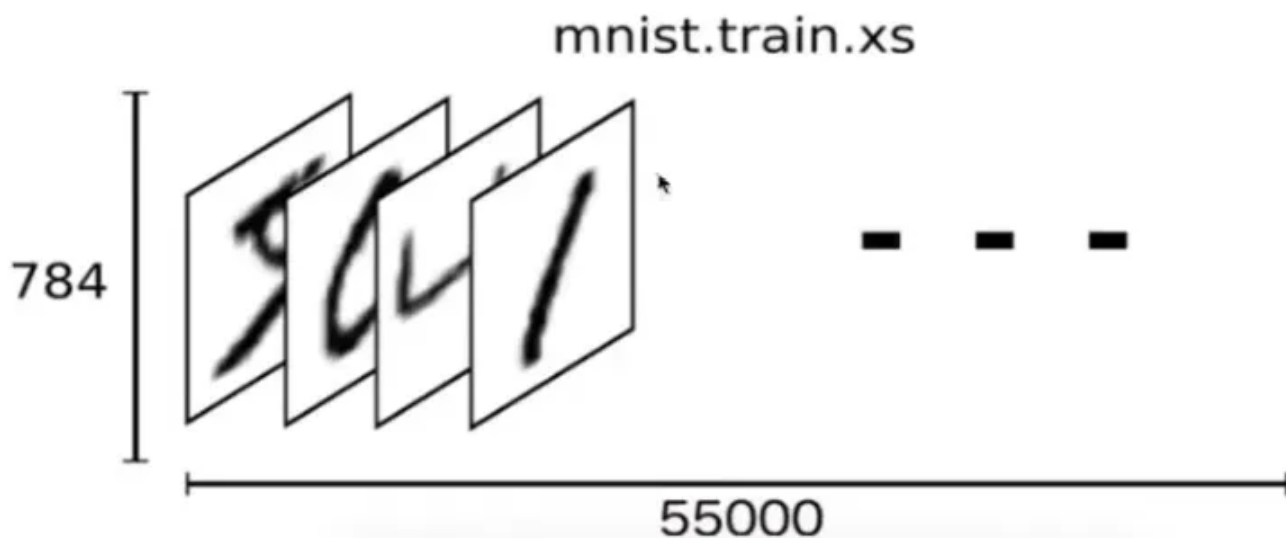
网址：<http://yann.lecun.com/exdb/mnist/>

特征值

Mnist数据集可以从官网下载，网址：<http://yann.lecun.com/exdb/mnist/> 下载下来的数据集被分成两部分：55000行的训练数据集（mnist.train）和10000行的测试数据集（mnist.test）。每一个MNIST数据单元有两部分组成：一张包含手写数字的图片和一个对应的标签。我们把这些图片设为“xs”，把这些标签设为“ys”。训练数据集和测试数据集都包含xs和ys，比如训练数据集的图片是 mnist.train.images，训练数据集的标签是 mnist.train.labels。



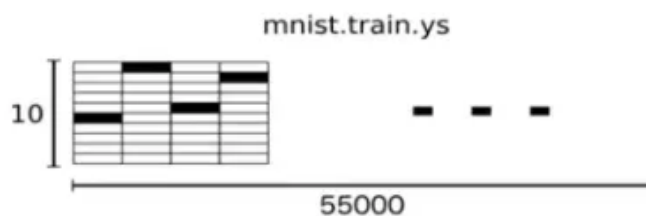
我们可以知道图片是黑白图片，每一张图片包含28像素X28像素。我们把这个数组展开成一个向量，长度是 $28 \times 28 = 784$ 。因此，在MNIST训练数据集中，`mnist.train.images` 是一个形状为 `[60000, 784]` 的张量。



目标值

MNIST中的每个图像都具有相应的标签，0到9之间的数字表示图像中绘制的数字。用的是one-hot编码

```
mn[0,0,0,1,0,0,0,0,0,0] mnist.train.labels [55000, 10]
```



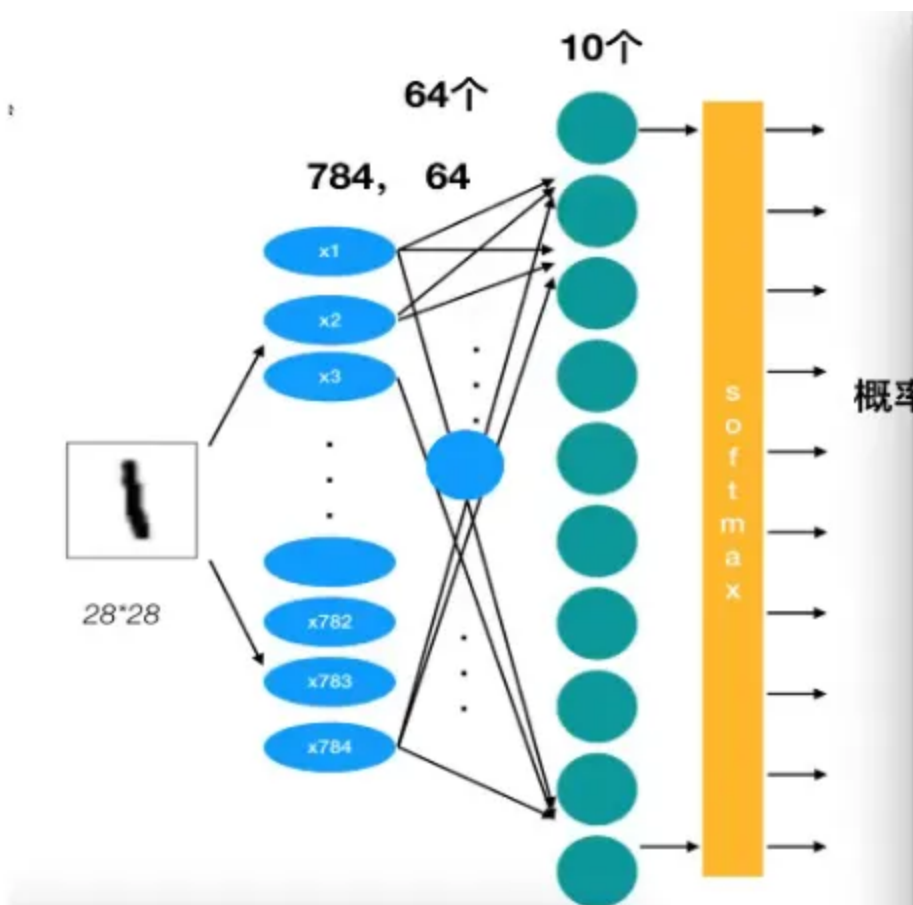
Mnist数据获取API

TensorFlow框架自带了获取这个数据集的接口，所以不需要自行读取。

- `from tensorflow.examples.tutorials.mnist import input_data`
 - `mnist = input_data.read_data_sets(FLAGS.data_dir, one_hot=True)`
 - `mnist.train.next_batch(100)`(提供批量获取功能)
 - `mnist.train.images`、`labels`
 - `mnist.test.images`、`labels`

网络设计

我们采取2个层，除了输入层之外。第一个隐层中64个神经元，最后一个输出层（全连接层）我们必须设置10个神经元的神经网络



隐层的参数: $W [784, 64]$, $b [64]$

输出层的参数: $W [64, 10]$, $b [10]$

全连接层计算

- `tf.matmul(a, b, name=None) + bias`
 - return: 全连接结果, 供交叉损失运算
- `tf.train.GradientDescentOptimizer(learning_rate)`
 - 梯度下降
 - `learning_rate`: 学习率
 - `method`:
 - `minimize(loss)`: 最小优化损失

前期确定事情与流程

- 确定网络结构以及形状
 - 第一层参数: 输入: `x [None, 784]` 权重: `[784, 64]` 偏置`[64]`, 输出`[None, 64]`
 - 第二层参数: 输入: `[None, 64]` 权重: `[64, 10]` 偏置`[10]`, 输出`[None, 10]`
- 流程:
 - 获取数据
 - 前向传播: 网络结构定义
 - 损失计算
 - 反向传播: 梯度下降优化
- 功能完善
 - 准确率计算
 - 添加Tensorboard观察变量、损失变化
 - 训练模型保存、模型存在加载模型进行预测

主网络搭建流程

- 获取数据

```
mnist = input_data.read_data_sets("./data/mnist/input_data/", one_hot=True)
```

- 定义数据占位符, mnist数据实时提供给placeholder

```

# 1、准备数据
# x [None, 784] y_true [None, 10]
with tf.variable_scope("mnist_data"):

    x = tf.placeholder(tf.float32, [None, 784])

    y_true = tf.placeholder(tf.int32, [None, 10])

```

- 两层神经网络结果计算
- 损失计算与优化

```

# 3、softmax回归以及交叉熵损失计算
with tf.variable_scope("softmax_crossentropy"):

    # labels:真实值 [None, 10] one_hot
    # logits:全脸层的输出 [None, 10]
    # 返回每个样本的损失组成的列表
    loss = tf.reduce_mean(tf.nn.softmax_cross_entropy_with_logits(labels=y_true,
                                                                    logits=y_predict))

# 4、梯度下降损失优化
with tf.variable_scope("optimizer"):

    # 学习率
    train_op = tf.train.GradientDescentOptimizer(0.1).minimize(loss)

```

```

import tensorflow as tf
from tensorflow.examples.tutorials.mnist import input_data

def fc_nn():
    # 网络结构流程确定
    # 1、获取数据
    mnist = input_data.read_data_sets("./data/mnist/input_data", one_hot=True)

    # 2、前向传播：建立网络结构
    # 定义网络输入特征
    with tf.variable_scope("data"):
        # 输入特征 x [None, 784] = [28, 28]
        X = tf.placeholder(tf.float32, [None, 784], name="X_data")

        # 目标值一定
        y = tf.placeholder(tf.int32, [None, 10], name="label")

    # 定义网络的隐层：64个神经元， 10个神经元
    with tf.variable_scope("hidden"):
        # 输入与参数计算，得到输出
        # [None, 784] * [784, 64] + [64] = [None, 64]
        # 参数值最好小一些
        # 初始化参数
        # 优化参数：必须使用变量去进行定义
        weight_hidden = tf.Variable(tf.random_normal([784, 64], mean=0.0, stddev=1.0),
                                    name="weight_hidden")

        bias_hidden = tf.Variable(tf.random_normal([64], mean=0.0, stddev=1.0),
                                   name="bias_hidden")

        # 进行一层前向传播计算
        A1 = tf.matmul(X, weight_hidden) + bias_hidden

```

```

# # 定义网络的输出层：10个神经元
with tf.variable_scope("fc"):
    # 初始化参数，计算
    # [None, 64] * [64, 10] + [10] = [None, 10]
    # 初始权重和偏置
    weight_fc = tf.Variable(tf.random_normal([64, 10], mean=0.0, stddev=1.0),
                            name="weight_fc")

    bias_fc = tf.Variable(tf.random_normal([10], mean=0.0, stddev=1.0),
                           name="bias_fc")

    # 前向传播计算
    y_predict = tf.matmul(A1, weight_fc) + bias_fc

```

```

# 3、计算损失
with tf.variable_scope("compute_loss"):

    # 目标值的softmax结果与真实值之间进行交叉熵损失计算
    # 计算出了所有样本的损失
    all_loss = tf.nn.softmax_cross_entropy_with_logits(labels=y,
                                                         logits=y_predict)

    # 计算损失,求平均值
    loss = tf.reduce_mean(all_loss)

# 4、反向传播: 优化
with tf.variable_scope("compute_loss"):

    # 梯度下降优化
    # 学习率
    train_op = tf.train.GradientDescentOptimizer(0.1).minimize(loss)

```

```

# 运行训练网络
with tf.Session() as sess:
    # 运行初始化变量(这一步在任何程序中都需要进行)
    sess.run(tf.global_variables_initializer())

    # 循环去进行训练, 指定步数去进行优化
    for i in range(2000):

        # 每一步训练指定50个样本去训练
        # 或者每一步吧所有样本都训练一遍
        mnist_x, mnist_y = mnist.train.next_batch(50)

        # 会话运行训练
        loss_run, _ = sess.run([loss, train_op], feed_dict={X: mnist_x, y:mnist_y})

        print("迭代第 %d 步, 损失为: %f" % (i, loss_run))

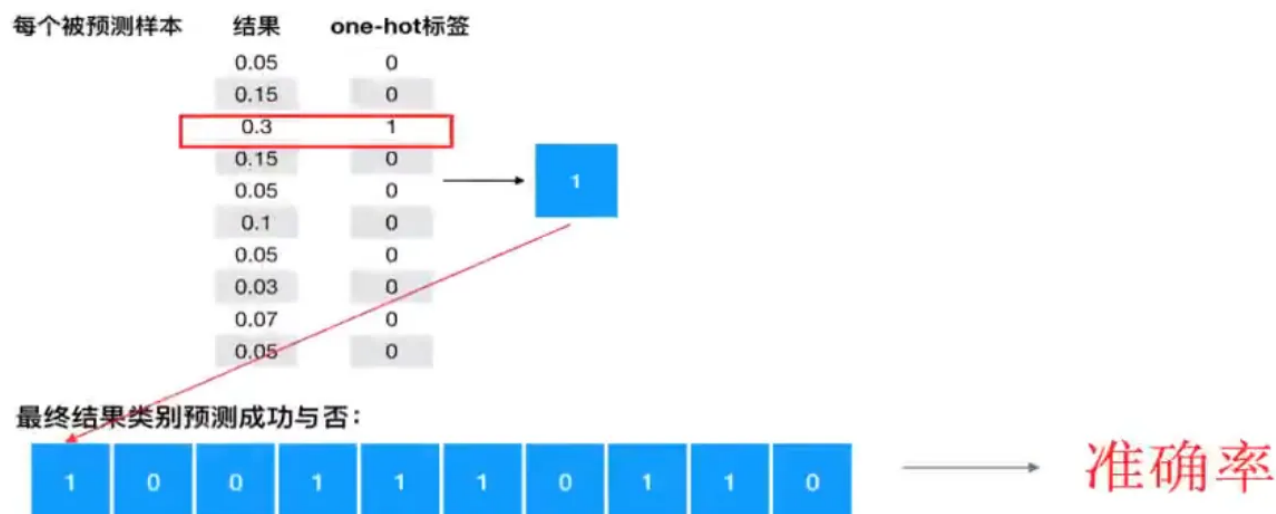
if __name__ == '__main__':
    fc_nn()

```

完善模型功能

1. 增加准确率计算
2. 增加变量tensorboard显示
3. 增加模型保存加载
4. 增加模型预测结果输出

如何计算准确率



- `equal_list = tf.equal(tf.argmax(y, 1), tf.argmax(y_label, 1))`
- `accuracy = tf.reduce_mean(tf.cast(equal_list, tf.float32))`

模型评估（计算准确性）

```
# 5、计算准确率
with tf.variable_scope("accuracy"):
    equal_list = tf.equal(tf.argmax(y_predict, 1), tf.argmax(y, 1))
    accuracy = tf.reduce_mean(tf.cast(equal_list, tf.float32))
```

```
第 1990 步的50个样本损失为: 0.936456 , 准确率为: 0.800000
第 1991 步的50个样本损失为: 0.339523 , 准确率为: 0.880000
第 1992 步的50个样本损失为: 0.220034 , 准确率为: 0.900000
第 1993 步的50个样本损失为: 0.600553 , 准确率为: 0.780000
第 1994 步的50个样本损失为: 0.533387 , 准确率为: 0.860000
第 1995 步的50个样本损失为: 0.367520 , 准确率为: 0.920000
第 1996 步的50个样本损失为: 0.919380 , 准确率为: 0.720000
第 1997 步的50个样本损失为: 1.172493 , 准确率为: 0.740000
第 1998 步的50个样本损失为: 0.971207 , 准确率为: 0.840000
第 1999 步的50个样本损失为: 0.966419 , 准确率为: 0.820000
```

增加变量tensorboard显示

在会话外当时增加以下代码

```

# (1)、收集要显示的变量
# 先收集损失和准确率
tf.summary.scalar("losses", loss)
tf.summary.scalar("acc", accuracy)

# 维度高的张量值
tf.summary.histogram("w1", weight_1)
tf.summary.histogram("b1", bias_1)
# 维度高的张量值
tf.summary.histogram("w2", weight_2)
tf.summary.histogram("b2", bias_2)

# 初始化变量op
init_op = tf.global_variables_initializer()

# (2)、合并所有变量op
merged = tf.summary.merge_all()

```

在会话当中去创建文件写入每次的变量值

```

# (1) 创建一个events文件实例
file_writer = tf.summary.FileWriter("./tmp/summary/", graph=sess.graph)

# 运行合变量op。写入事件文件当中
summary = sess.run(merged, feed_dict={x: mnist_x, y_true: mnist_y})

file_writer.add_summary(summary, i)

```

```

# 1、收集变量
tf.summary.scalar('losses', loss)
tf.summary.scalar('acc', accuracy)

tf.summary.histogram('weight_hidden', weight_hidden)
tf.summary.histogram('bias_hidden', bias_hidden)
tf.summary.histogram('weight_fc', weight_fc)
tf.summary.histogram('bias_fc', bias_fc)

```

增加模型保存加载

创建Saver，然后保存

```
# 创建模型保存和加载
saver = tf.train.Saver()

# 每隔100步保存一次模型
if i % 100 == 0:

    saver.save(sess, "./tmp/modelckpt/fc_nn_model")
```

在训练之前加载模型

```
# 加载模型
if os.path.exists("./tmp/modelckpt/checkpoint"):

    saver.restore(sess, "./tmp/modelckpt/fc_nn_model")
```

增加模型预测结果输出

增加标志位

```
tf.app.flags.DEFINE_integer("is_train", 1, "指定是否是训练模型，还是拿数据去预测")
FLAGS = tf.app.flags.FLAGS
```

然后判断是否训练，如果不是训练就直接预测，利用tf.argmax对样本的真实目标y_true，和预测的目标值y_predict求出最大的位置

```
# 如果不是训练，我们就去进行预测测试集数据
for i in range(100):

    # 每次拿一个样本预测
    mnist_x, mnist_y = mnist.test.next_batch(1)

    print("第%d个样本的真实值为: %d, 模型预测结果为: %d" % (
        i+1,
        tf.argmax(sess.run(y_true, feed_dict={x: mnist_x,
        tf.argmax(sess.run(y_predict, feed_dict={x: mnist_x,
        )
        )
```

- 准确率计算
 - `tf.argmax, tf.equal, tf.reduce_mean`
- 添加Tensorboard观察变量、损失变化
 - `tf.summary`
 - `tf.summary.FileWriter("./tmp/summary/", graph=sess.graph)`
 - `file_writer.add_summary(summary, i)`
- 训练模型保存、模型存在加载模型进行预测
 - `tf.train.Saver`
 - `save`
 - `restore`

调整学习率调整网络参数带来的问题

- 如果我们对网络当中的学习率进行修改，也就是一开始我们并不会知道学习率项哪些值，也并不知道调整网络的参数大小带来的影响（第一部分第四节）。

假设学习率调整到1,2

假设参数调整到比较大的值，几十，几百

总结：参数调整了之后可能没影响，是因为网络较小，可能并不会适成后面所介绍的梯度消失或者棒皮爆炸。

总结

- 掌握softmax公式以及特点
- `tensorflow.examples.tutorials.mnist.input_data` 获取Mnist数据
- `tf.matmul(a, b, name=None)`实现全连接层计算
- `tf.train.GradientDescentOptimizer(learning_rate).minimize(loss)`实现梯度下降优化